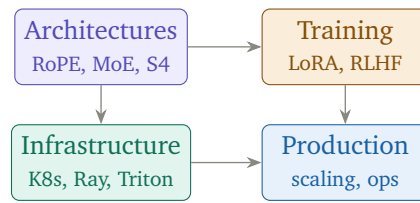


BOOK 3

Model Architectures, Training Techniques, and Applied Infrastructure

From positional encodings to production platforms



The third volume, completing
LLM Inference • *LLM Systems Engineering*

Carmel Prosper Sagbo

Research Associate & ML Infrastructure Engineer
CyLab-Africa / Upanzi Network
Carnegie Mellon University Africa, Kigali

June 2026

For internal study and reference

Contents

About This Volume	v
I Model Architectures and Training Techniques	1
1 Positional Encodings	3
1.1 Why Position Must Be Added	3
1.2 Sinusoidal Embeddings	3
1.3 Learned and Relative Positional Embeddings	3
1.4 RoPE: Rotary Position Embedding	4
2 The Architecture Zoo	5
2.1 The Sequence-Model Family Tree	5
2.2 RNN vs Transformer vs S4	5
2.3 Notable Variants	6
2.4 Causal vs Cross Attention	6
3 Tokenisation and Scaling Laws	7
3.1 Tokenisation	7
3.2 Scaling Laws	7
4 The Training Pipeline	9
4.1 The Three Stages	9
4.1.1 Pretraining	9
4.1.2 Supervised Fine-Tuning (SFT)	9
4.1.3 Alignment	9
4.2 Parameter-Efficient Fine-Tuning: LoRA	9
5 Alignment: RLHF and Beyond	11
5.1 RLHF: The Three Steps	11
5.2 DPO and Simpler Alternatives	11
5.3 Where Reasoning Models Fit	12
6 The Numerical and Optimization Toolkit	13
6.1 Floating-Point Representation	13
6.2 Exploding and Vanishing Gradients	13
6.3 Fitting Training into Memory	14
6.3.1 Gradient Accumulation	14
6.3.2 Gradient Checkpointing	14

6.3.3 Numerical Precision Tricks	14
6.4 Frameworks and JIT Compilation	15
II Applied LLM Infrastructure	17
7 Kubernetes for Inference	19
7.1 Why an Orchestrator	19
7.2 The Core Objects	19
7.3 GPU Scheduling	20
8 Serving Platforms	21
8.1 The Landscape	21
8.2 Triton in Brief	21
8.3 Ray Serve and Composition	21
8.4 Multi-Tenant Serving	22
9 Autoscaling and Observability	23
9.1 Autoscaling	23
9.2 Observability	23
9.2.1 OpenTelemetry	24
10 Distributed RAG and Agent Orchestration	25
10.1 Distributed RAG	25
10.2 Agent Orchestration	25
11 Production Incidents	27
11.1 The Common Failure Modes	27
11.2 An Incident Mindset	27
III Hands-On Labs	29
12 Architecture, Training, and Platform Labs	31
13 Lab 1 — Fine-Tune with LoRA	33
14 Lab 2 — Compare Positional Encodings	35
15 Lab 3 — Deploy on Kubernetes	37
16 Lab 4 — Autoscale Under Load	39
17 Lab 5 — Trace a RAG Request	41
Closing: The Complete Arc	43

About This Volume

Books 1 and 2 followed one thread: how a transformer runs, and how the hardware beneath it runs. They were deliberately narrow — inference, serving, and the systems supporting them. Two things were intentionally left out.

First, **model variety and how models are made**. Books 1 and 2 assumed a standard decoder-only transformer with weights that already exist. They never explained the other architectures you will meet (state-space models, recurrent transformers, encoder-decoders), the positional-encoding designs that distinguish them, or the training methods that produce and adapt them (pretraining, fine-tuning, LoRA, RLHF).

Second, **the production platform**. Book 2 reached the single serving engine. It stopped before the cluster: how you run that engine on Kubernetes, autoscale it, observe it, and orchestrate agents and RAG across many machines.

In plain terms Books 1 and 2 explained one car and its engine in depth. This book does two things: it walks the showroom (the other architectures and how any of them is built and tuned), and it builds the highway system the cars actually drive on (the production platform). It is two halves under one cover.

Part A — *Model Architectures and Training Techniques* — fills the model-side gaps. Part B — *Applied LLM Infrastructure* — builds the production platform. The same illustrated, plain-language, lab-driven style as Books 1 and 2 continues throughout, and cross-references point back when a concept was introduced earlier.

PART I

Model Architectures and Training Techniques

Positional Encodings

A transformer’s attention is, by itself, blind to order: shuffle the input tokens and self-attention computes the same thing. Positional encodings inject the missing sense of *where*. This chapter traces the three generations, because the differences explain real model behaviour like context-length limits.

1.1 Why Position Must Be Added

Attention (Book 1, Chapter 1) computes a weighted sum over all tokens. Nothing in that sum depends on a token’s position. “dog bites man” and “man bites dog” would be identical to a position-blind model. Positional encoding fixes this by making each token’s representation depend on where it sits.

In plain terms Attention treats the input like a bag of words — it sees which words are present and how they relate, but not their order. Positional encoding is the stamp that says “this word was 5th.” Without it, a sentence and its anagram look the same to the model.

1.2 Sinusoidal Embeddings

The original transformer added a fixed pattern of sine and cosine waves of different frequencies to each token embedding. Position pos , dimension i :

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Each dimension oscillates at a different wavelength, so the combination is a unique fingerprint for every position. Being fixed (not learned), it can in principle extend to positions never seen in training.

1.3 Learned and Relative Positional Embeddings

Learned absolute embeddings replace the fixed waves with a trainable vector per position. Simple and effective, but capped at the maximum position seen in training — they cannot extrapolate.

Relative positional embeddings encode the *distance* between two tokens rather than their absolute slots. What matters for “the cat sat” is that “sat” is two tokens after “the,” regardless of where the phrase appears. This generalizes far better to longer sequences and shifts.



1.4 RoPE: Rotary Position Embedding

RoPE is the modern default (Llama, Mistral, Qwen), and Book 1 named it without explaining it. The idea: instead of *adding* a positional vector, RoPE *rotates* each query and key vector by an angle proportional to its position.

Because a dot product between two rotated vectors depends only on the difference of their rotation angles, the attention score between two tokens naturally encodes their *relative* distance — RoPE gets the generalization of relative encodings through a clean geometric trick applied inside attention itself.

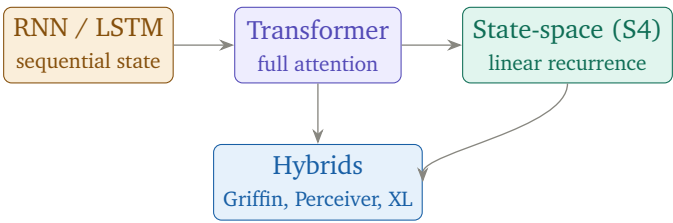
RoPE’s rotation-by-position is why context-extension tricks work. Methods like positional interpolation and YaRN rescale the rotation frequencies so a model trained at 4k tokens can run at 32k or beyond. When you read that a model’s context was “extended,” it almost always means its RoPE frequencies were adjusted. This connects directly to the KV-cache and context-length costs of Books 1 and 2.

In plain terms Think of each token’s vector as a clock hand. RoPE turns the hand by an amount set by the token’s position. When two tokens compare (a dot product), what registers is the *angle between their hands* — which is just their distance apart. Position becomes geometry.

The Architecture Zoo

Books 1 and 2 assumed a decoder-only transformer. It is dominant, but not alone. Understanding the alternatives clarifies *why* the transformer won and where it is challenged.

2.1 The Sequence-Model Family Tree



2.2 RNN vs Transformer vs S4

Family	How it handles sequence	Cost / tradeoff
RNN / LSTM	maintains a hidden state, processes one token at a time, carrying memory forward	$O(N)$ time, $O(1)$ memory per step, but <i>sequential</i> — cannot parallelize training over the sequence, and long-range memory fades
Transformer	every token attends to every other token directly	$O(N^2)$ attention, but fully <i>parallel</i> in training; long-range links are direct
State-space (S4, Mamba)	a structured linear recurrence that can be computed as a convolution	near $O(N)$, parallelizable in training <i>and</i> efficient at inference; strong on very long sequences

The transformer won because it is *parallelizable in training* — the whole sequence processes at once, exploiting the GPU (Book 2, Part I). The RNN’s fatal flaw was not accuracy but that its sequential nature wasted the hardware. State-space models are the current serious challenger: they aim to keep the transformer’s training parallelism while regaining the RNN’s linear inference cost — directly attacking the KV-cache and $O(N^2)$ problems from Books 1 and 2.

In plain terms An RNN reads a book word by word, remembering a running summary — cheap but forgetful and slow to train. A transformer spreads the whole page on a desk and draws lines between every pair of words — powerful but the lines grow as the square of the page length. State-space models try to read like the RNN but train like the transformer.

2.3 Notable Variants

- **Transformer-XL**: adds a recurrence *between* segments — it caches hidden states from the previous segment so attention can reach back beyond the current window. An early answer to the fixed-context-length problem.
- **Griffin**: a hybrid interleaving gated linear recurrences with local attention, aiming for transformer-quality at lower inference cost on long sequences.
- **Perceiver**: tackles very large or multimodal inputs by first cross-attending the input into a small fixed-size set of latent vectors, so the expensive self-attention runs on the small latent set rather than the huge input — decoupling compute from input size.

2.4 Causal vs Cross Attention

Book 1 explained self-attention. Two variants complete the picture.



- **Causal attention** masks the future: a token may attend only to itself and earlier tokens. This is what makes a decoder generate left-to-right — it cannot peek ahead. Every model in Books 1 and 2 used causal attention; the mask is the one detail those books left implicit.
- **Cross attention** lets one sequence attend to a *different* sequence: the queries come from one place (e.g. the decoder), the keys and values from another (e.g. an encoder's output, or an image). It is the bridge in encoder-decoder models (translation, the Perceiver's latents) and in many multimodal designs.

Self, causal, and cross attention are the same operation (Book 1's Q/K/V) with different rules about *who may look at whom*. Self: everyone sees everyone. Causal: everyone sees only the past. Cross: one group looks at another. The machinery never changes — only the mask and the source of K and V.

Tokenisation and Scaling Laws

Two foundations Books 1 and 2 used without unpacking: how text becomes tokens, and how we decide how big a model should be.

3.1 Tokenisation

Models do not see characters or words; they see *tokens*, integer IDs from a fixed vocabulary. The dominant scheme is **Byte-Pair Encoding (BPE)** (and variants like WordPiece, Unigram, SentencePiece).

BPE starts from individual bytes and repeatedly merges the most frequent adjacent pair into a new token, until the vocabulary reaches a target size (often 32k–128k). Common words become single tokens; rare words split into sub-word pieces; anything can be represented because it bottoms out at bytes.



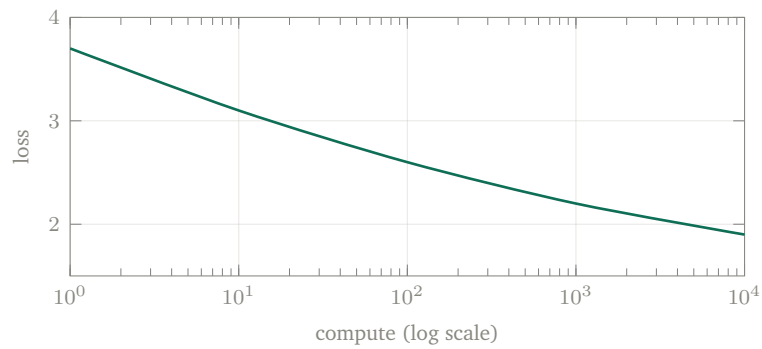
In plain terms A tokenizer is a phrasebook the model memorized. Frequent words get their own entry; unusual words are spelled out from smaller fragments. “the” is one token; an unusual surname might be four. This is why token counts (and therefore cost, Book 2) do not equal word counts.

Tokenisation quirks cause real bugs: numbers split inconsistently (hurting arithmetic), non-English text often uses more tokens per word (raising cost for those languages), and trailing-space differences can change outputs. When a model behaves oddly on a specific string, check how it tokenizes first.

3.2 Scaling Laws

How big should a model be, and how much data should it see? Scaling laws answer empirically: model loss falls as a smooth power law in three quantities — parameters, data, and compute.

The landmark result (Chinchilla) found that, for a fixed compute budget, most large models were *undertrained*: they had too many parameters and too few training tokens. The compute-optimal rule of thumb that emerged is to scale parameters and training tokens *together*, roughly 20 tokens per parameter.



Scaling laws are why the field could justify spending millions on a single training run: the loss curve is *predictable*. You can extrapolate from small runs to estimate the loss of a much larger one before committing the compute. The “LLM scaling factor” is this relationship — how much better the model gets per additional unit of compute, and how to split that compute between a bigger model and more data.

The Training Pipeline

Book 2 covered the mechanics of *a* training step. This chapter covers the full pipeline that turns raw text into a helpful assistant: pretraining, fine-tuning, parameter-efficient adaptation, and alignment.

4.1 The Three Stages



4.1.1 Pretraining

The model learns language by predicting the next token over a vast corpus (trillions of tokens). The objective is simple — minimize next-token loss — but it is where almost all knowledge and capability is acquired. This is the expensive stage from the scaling-law chapter. The output is a *base model*: fluent, knowledgeable, but not yet good at following instructions.

4.1.2 Supervised Fine-Tuning (SFT)

The base model is fine-tuned on curated (instruction, response) pairs so it learns to *follow instructions* rather than merely continue text. Same next-token objective, but on demonstration data. This produces an instruct model. Far cheaper than pretraining — thousands to millions of examples, not trillions of tokens.

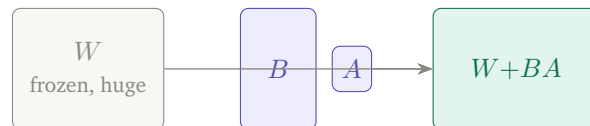
4.1.3 Alignment

A final stage shapes the model toward helpful, harmless, honest behaviour and human preferences — covered in its own chapter below.

4.2 Parameter-Efficient Fine-Tuning: LoRA

Full fine-tuning updates every weight — expensive in memory (Book 2: gradients + optimizer state for all parameters). **LoRA (Low-Rank Adaptation)** freezes the original weights and learns a small low-rank update instead.

For a weight matrix W , LoRA learns two small matrices A and B whose product is added: $W' = W + BA$, where B and A are far smaller (low rank r). Only A and B train; W stays frozen.



LoRA can cut trainable parameters by 100–1000 $\times$, so a model that needs a cluster to fully fine-tune can be adapted on a single GPU. **QLoRA** goes further: quantize the frozen base to 4-bit (Book 2’s quantization) and train LoRA adapters on top, fitting fine-tuning of a 70B model onto one GPU. Adapters are tiny and swappable — one base model, many task-specific adapters loaded on demand.

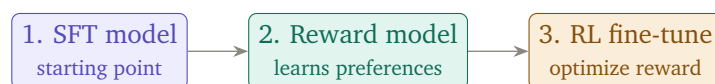
In plain terms Full fine-tuning rewrites the whole textbook. LoRA leaves the textbook untouched and clips in a few annotated sticky notes. The notes are small, cheap to write, and you can swap different note sets for different tasks — all sharing the same underlying book.

Alignment: RLHF and Beyond

Pretraining and SFT make a model capable and instruction-following. Alignment makes it prefer the responses humans actually want. This is where much of the gap between a raw model and a polished assistant is closed.

5.1 RLHF: The Three Steps

Reinforcement Learning from Human Feedback turns human preferences into a training signal.



1. Start from the SFT model.
2. **Train a reward model:** show humans pairs of responses, record which they prefer, and train a model to predict that preference as a scalar reward. The reward model is a learned stand-in for human judgement.
3. **Reinforcement-learn against it:** fine-tune the SFT model (classically with PPO) to maximize the reward model's score, while a KL penalty keeps it from drifting too far from the original and producing gibberish that games the reward.

In plain terms You cannot write a rule for “a good answer.” So instead you show people pairs of answers and ask which is better, train a judge model to imitate those choices, then coach the assistant to score well with that judge — without letting it wander so far it starts gaming the judge. That coaching loop is RLHF.

5.2 DPO and Simpler Alternatives

PPO-based RLHF is powerful but fiddly — it juggles a policy, a reward model, and a reference model at once. **Direct Preference Optimization (DPO)** achieves similar results by optimizing the preference data directly, skipping the separate reward model and the RL loop. It reframes alignment as a simple classification-style loss on preferred-vs-rejected pairs, which is far more stable and cheaper to run — which is why many recent open models use DPO or its relatives.

The trajectory is toward simplicity: RLHF (reward model + PPO) → DPO (direct on preferences) → a family of even lighter methods. The constant is the data — pairs of responses with a human

(or AI) preference between them. Alignment quality is bounded by the quality of that preference data more than by the choice of algorithm.

5.3 Where Reasoning Models Fit

The reasoning models of Book 1 are trained with a related but distinct signal: reinforcement learning against *verifiable* rewards (did the math check out, did the code pass the tests) rather than human preference. This is why they excel at domains with checkable answers — the reward is objective, not a learned approximation of taste.

The Numerical and Optimization Toolkit

This chapter collects the training-time techniques that Books 1 and 2 used implicitly or skipped: how numbers are represented, how gradients misbehave and how to tame them, the tricks that fit big training into limited memory, and the framework/compilation layer underneath it all.

6.1 Floating-Point Representation

Every number in a model is a floating-point value: a sign, an exponent (range), and a mantissa (precision).



Format	Bits	Exp / Mant	Character
FP32	32	8 / 23	full precision, slow, memory-heavy
FP16	16	5 / 10	precise but narrow range; can overflow/underflow
BF16	16	8 / 7	FP32's range, less precision; the training default
INT8 / INT4	8 / 4	integer	inference quantization (Books 1–2)

The FP16-vs-BF16 choice is entirely about the exponent. BF16 keeps FP32's 8 exponent bits, so it represents the same range of magnitudes and rarely overflows — it just carries fewer significant digits. For gradients that span huge magnitude ranges, range matters more than precision, which is why BF16 became the training standard (Book 2, mixed precision).

In plain terms A float is scientific notation: the exponent says how big (the power of ten), the mantissa says the exact digits. FP16 has few exponent bits, so very large or very small numbers fall off the end. BF16 sacrifices some digits to keep the wide range — safer for training, where gradients can be tiny or huge.

6.2 Exploding and Vanishing Gradients

Backpropagation multiplies many gradients together through the layers. If they are consistently > 1 , the product explodes toward infinity; if consistently < 1 , it vanishes toward zero. Either way, learning breaks — exploding gradients produce NaNs, vanishing gradients freeze the early layers.



The defences, several of which are already in the transformer:

- **Residual connections:** the skip-paths in every transformer block give gradients a direct route backward, the single biggest reason deep transformers train at all.
- **Layer normalization:** keeps activations in a stable range layer to layer.
- **Gradient clipping:** cap the gradient’s norm at a threshold before the update, directly preventing explosions. A one-line safeguard in nearly every training loop.
- **Careful initialization:** scale initial weights so signals neither grow nor shrink on average.

If training loss suddenly becomes NaN, exploding gradients are the first suspect — add or tighten gradient clipping. If the loss plateaus immediately and the early layers never change, suspect vanishing gradients or a learning rate too small.

6.3 Fitting Training into Memory

Book 2 showed training needs several times the memory of the weights. Three techniques claw it back.

6.3.1 Gradient Accumulation

Want a large batch but it does not fit? Run several small *micro-batches*, sum their gradients, and only then update. This simulates a large batch’s gradient with a small batch’s memory, at the cost of more forward/backward passes per update.

6.3.2 Gradient Checkpointing

The backward pass needs the activations saved during the forward pass (Book 2). Gradient checkpointing saves only a few and *recomputes* the rest on demand during backprop — trading extra compute for a large drop in activation memory. Often the difference between a model fitting on your GPU or not.



6.3.3 Numerical Precision Tricks

Beyond format choice: *loss scaling* multiplies the loss before backprop so small FP16 gradients do not underflow to zero, then divides back before the update. *Stochastic rounding* rounds randomly in proportion to distance, so tiny updates are not always lost to rounding. These keep low-precision training accurate.

Gradient accumulation, gradient checkpointing, and mixed precision are the three levers that let a model train on less hardware than its raw memory math suggests. They are the training-time cousins of the inference-time memory tricks (KV cache, quantization, PagedAttention) — the same theme of trading compute or precision for memory that runs through all three books.

6.4 Frameworks and JIT Compilation

The model math runs on top of a framework. The three that matter:

Framework	Character
PyTorch	define-by-run (eager); dominant in research for its flexibility and debuggability. Used in this book's labs.
TensorFlow	define-then-run historically; strong production tooling and deployment story.
JAX	functional, composable transforms (<code>grad</code> , <code>vmap</code> , <code>jit</code>); favoured for large-scale research and TPUs.

JIT compilation bridges flexible Python and fast hardware. Eager execution runs operations one at a time — easy to debug, but it cannot fuse or optimize across operations. A JIT compiler (`torch.compile`, JAX's `jit`, XLA) traces the computation into a graph, then fuses kernels, eliminates overhead, and optimizes the whole graph before running.



JIT fusion is the same idea as Book 2's kernel fusion, raised to the framework level: combine many small operations so intermediate results stay in fast memory instead of round-tripping through HBM. `torch.compile` can give large speedups for free precisely because it automates the fusion a kernel engineer would otherwise do by hand.

PART II

Applied LLM Infrastructure

Kubernetes for Inference

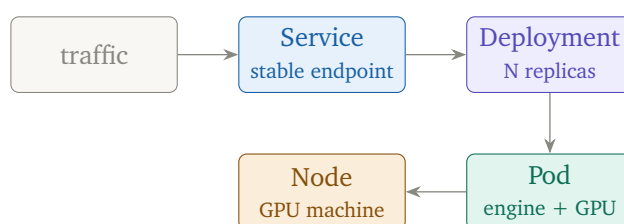
Book 2 reached the single serving engine on one machine. Production runs many engines across many machines, and Kubernetes is the de facto system for orchestrating them. This chapter assumes no Kubernetes background.

7.1 Why an Orchestrator

A real deployment is not one vLLM process. It is many replicas across many GPU nodes, that must be started, health-checked, restarted on failure, load-balanced, and scaled up and down with demand. Doing this by hand does not survive contact with production. An orchestrator automates it.

In plain terms Running one model server by hand is like running one food stall. A production deployment is a chain of hundreds of stalls that open and close with the crowd, replace broken equipment automatically, and route each customer to a free counter. Kubernetes is the manager that runs the whole chain so you do not stand at every stall.

7.2 The Core Objects



- **Pod:** the smallest unit — one or more containers running together, here your inference engine plus its GPU.
- **Deployment:** declares how many identical Pod replicas should exist and keeps that many running, replacing any that die.
- **Service:** a stable network address that load-balances across the Pods, so callers do not track individual replicas.
- **Node:** a physical or virtual machine; GPU nodes carry the accelerators.

7.3 GPU Scheduling

GPUs are scarce and expensive, so scheduling them well is the heart of LLM infrastructure on Kubernetes.

- **Resource requests:** a Pod declares it needs, say, one GPU; the scheduler places it only on a node with a free one.
- **Node selectors / taints:** pin GPU workloads to GPU nodes and keep ordinary workloads off them.
- **Fractional GPUs (MIG, time-slicing):** a large GPU can be partitioned so several small models share it — important because a 3B model wastes an H100 alone.

GPU scheduling has a sharp failure mode: a Pod requesting a GPU that no node can satisfy stays *Pending* forever, silently. Always alert on Pending GPU Pods and on GPU utilization — an idle but reserved GPU is pure cost (Book 2's cost model), and a Pending Pod is unserved traffic.

Serving Platforms

Kubernetes provides the primitives; serving platforms build LLM-aware serving on top of them, adding model loading, batching integration, autoscaling, and traffic management you would otherwise hand-build.

8.1 The Landscape

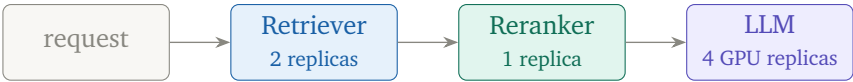
Platform	What it adds
Triton Inference Server	NVIDIA's high-performance server; multi-framework, dynamic batching, model ensembles, and concurrent model execution on one GPU.
Ray Serve	Python-native serving on the Ray distributed runtime; flexible composition of models and business logic, good for multi-step and agentic pipelines.
KServe	Kubernetes-native model serving with a standard inference protocol, autoscaling (including scale-to-zero), and canary rollouts as first-class features.

8.2 Triton in Brief

Triton sits closest to the metal. It loads models from many frameworks, applies its own *dynamic batching* (the continuous-batching idea of Book 1, at the server level), and can run several models concurrently on one GPU — the fractional-GPU idea made practical. It exposes standard HTTP/gRPC endpoints and rich per-model metrics.

8.3 Ray Serve and Composition

Ray Serve's strength is *composition*. A request can flow through several deployments — a retriever, a reranker, an LLM, a post-processor — each scaled independently, each a Python class.



This is Book 1's RAG pipeline, deployed: each stage becomes an independently scaled service. The retriever is cheap and CPU-bound; the LLM is expensive and GPU-bound. Scaling them separately — many retrievers, fewer LLM replicas — is how you avoid paying GPU prices for CPU work.

Composition platforms make that separation natural.

8.4 Multi-Tenant Serving

Serving many models or many customers from shared hardware introduces its own problems:

- **Model multiplexing:** load several models (or LoRA adapters, Part A) on one GPU and route each request to the right one — adapters make this cheap because the base model is shared.
- **Isolation and fairness:** prevent one tenant's burst from starving others, via per-tenant rate limits and quotas.
- **Cold starts:** loading a large model takes time; keep hot replicas for popular models and accept a cold-start delay for rare ones, or use scale-to-zero with a warm pool.

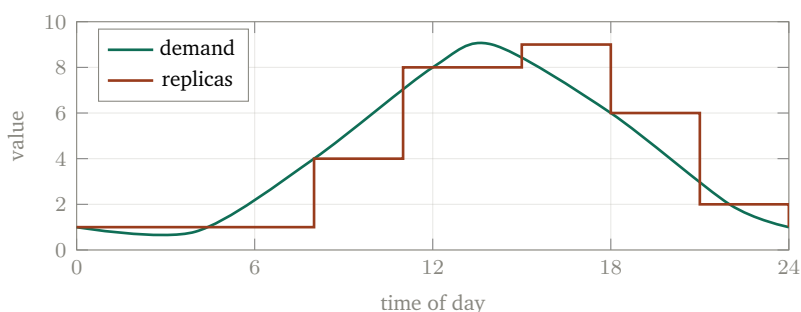
In plain terms Multi-tenancy is a shared kitchen serving many restaurants at once. You want to reuse the big shared oven (the base model) across menus (LoRA adapters), stop one busy restaurant from hogging everything (fairness), and not keep every rare dish's station heated all day (cold starts). The platform manages that sharing.

Autoscaling and Observability

A platform that cannot scale with demand or tell you what it is doing is not production-ready. These two capabilities turn a deployment into an operable service.

9.1 Autoscaling

Demand varies; fixed capacity either wastes money at the trough or fails at the peak. Autoscaling adjusts replica count to load.



- **Horizontal scaling:** add or remove replicas. The main lever for LLM serving.
- **Scaling signal:** scale on a meaningful metric. CPU is wrong for GPU work; scale on queue depth, GPU utilization, or requests-per-replica.
- **Scale-to-zero:** for rare models, drop to zero replicas and pay nothing until a request arrives — accepting a cold-start delay (Part B, multi-tenancy).

LLM autoscaling has a brutal constraint: spinning up a new replica means loading tens of gigabytes of weights into a fresh GPU, which can take minutes. By the time the replica is ready, the spike may be over. This is why LLM serving keeps warm headroom and scales on *leading* signals (rising queue depth) rather than reacting after latency has already degraded.

9.2 Observability

You cannot operate what you cannot see. Observability rests on three pillars.



- **Metrics:** aggregate numbers — TTFT percentiles, tokens/sec, GPU utilization, queue depth, cost per token. Prometheus collects, Grafana visualizes.
- **Logs:** discrete events, including errors and slow requests.
- **Traces:** the path of a single request across services, showing where time went — retriever, reranker, LLM, post-processing.

9.2.1 OpenTelemetry

OpenTelemetry is the vendor-neutral standard for emitting metrics, logs, and traces. Instrument once with its SDK; export anywhere (Prometheus, Jaeger, a hosted backend). For a multi-stage LLM pipeline this is essential: a distributed trace shows whether a slow response was the retriever, the queue, or decode.

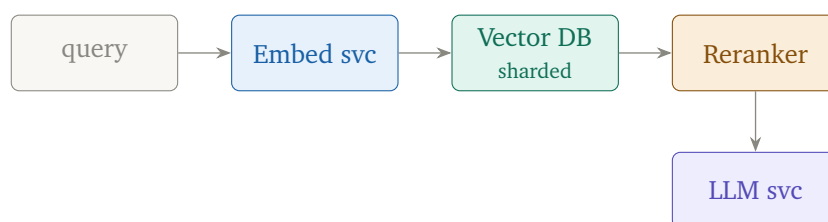
The metrics that matter were defined back in Books 1 and 2 — TTFT, tokens/sec, percentiles, cost per token, the load-vs-latency knee. Observability is how you actually *watch* them in production. A trace that attributes a 2-second response to 1.8 seconds of queue wait tells you to add capacity, not to optimize the kernel — exactly the lifecycle distinction from Book 1.

Distributed RAG and Agent Orchestration

The application patterns of Book 1 — RAG and agents — become distributed systems at scale. This chapter covers what changes when they no longer fit on one machine.

10.1 Distributed RAG

A production RAG system separates into independently scaled, independently failing services.



At scale the vector database is *sharded* across machines (millions to billions of vectors), the embedding and reranking models are their own GPU or CPU services, and the index must be kept fresh as documents change — an ingestion pipeline running continuously alongside serving. Each hop adds latency and a failure mode, so caching (embeddings, frequent queries) and graceful degradation (return best-effort results if the reranker is down) become design requirements.

10.2 Agent Orchestration

A single agent (Book 2) is a loop of LLM calls and tool uses. Running agents in production means orchestrating many such loops reliably.

- **State management:** an agent's trajectory is long-lived; its state (plan, memory, intermediate results) must persist across calls and survive a worker crash.
- **Tool execution as services:** tools (code execution, search, database access) are themselves services with their own scaling, sandboxing, and failure handling.
- **Concurrency and cost control:** a fan-out of parallel reasoning paths (Book 1's best-of-N, tree search) multiplies cost; orchestration enforces budgets and timeouts.
- **Durability:** long agent runs need checkpointing so a failure halfway does not discard an hour of work.

In plain terms One agent is a worker following a checklist, picking up tools as needed. Orchestrating agents is running a whole workforce: keeping each worker's progress saved if they trip, sharing the tool shed safely, capping how much overtime anyone runs up, and resuming interrupted jobs rather than starting over.

Distributed RAG and agent orchestration are where every layer of all three books converges in one request: a query hits a load balancer (Part B), fans out to sharded retrieval and GPU-served models (Books 1–2), each running optimized kernels on managed GPUs (Book 2), with the whole trajectory traced and cost-accounted (Part B). Operating this well is the full ML-systems-engineering job.

Production Incidents

Theory meets reality during incidents. This short chapter catalogues the failures that actually happen in LLM serving, framed so you recognize them fast.

11.1 The Common Failure Modes

Symptom	Likely cause and fix
Out-of-memory on long requests	KV cache outgrew VRAM (Books 1–2). Cap max context, enable KV quantization, or add memory.
Latency spikes under load	queue saturation past the knee (Book 2). Add replicas, tighten admission control.
Pending GPU pods	no schedulable GPU (Part B). Check node pool, requests, taints.
Throughput collapse after deploy	a new long-prompt pattern causing prefill–decode interference (Book 1). Consider prefill–decode disaggregation.
NaN losses (training)	exploding gradients (Part A). Add or tighten gradient clipping.
Quality regression after quantizing	outlier-weight damage at low bit-width (Book 2). Use AWQ-style protection or a higher bit-width.
Rising cost per token	utilization dropped — small batches or idle reserved GPUs (Book 2). Rebalance batching and replica count.

The recurring lesson from real incidents: the failure is rarely where the symptom appears. High latency is often a queue problem, not a model problem; OOM is often a context-length problem, not a leak; a quality drop is often quantization, not the prompt. Good observability (previous chapter) is what lets you find the real cause instead of guessing.

11.2 An Incident Mindset

1. **Look at the trace first.** Where did the time or the failure actually occur — queue, retrieval, prefill, decode, a tool?
2. **Separate model from system.** Is the model wrong, or is the system around it failing to deliver the model’s output on time?
3. **Check the cheap causes before the deep ones.** Pending pods, a bad deploy, a config change, and capacity explain most incidents before any kernel-level investigation.

4. **Write it down.** A short, blameless incident report (timeline, cause, fix, prevention) is how a team stops repeating failures.

PART III

Hands-On Labs

Architecture, Training, and Platform Labs

These labs close the three-book arc: a couple on the model side from Part A, the rest on the platform side from Part B.

In plain terms The Part A labs run on a single GPU or even CPU. The Part B labs want a Kubernetes cluster — a local one (kind, minikube, or k3s) is enough to see every concept; you do not need a cloud GPU fleet to learn the mechanics.

Lab 1 — Fine-Tune with LoRA

Reinforces: Part A (LoRA, training pipeline).

Hypothesis: LoRA adapts a model on a fraction of full fine-tuning’s memory.

```
from peft import LoraConfig, get_peft_model
cfg = LoraConfig(r=8, lora_alpha=16,
                 target_modules=["q_proj", "v_proj"])
model = get_peft_model(base_model, cfg)
model.print_trainable_parameters() # tiny %
```

Procedure: fine-tune a small base model on a few hundred instruction pairs with LoRA. Log trainable-parameter count and peak VRAM; compare against an attempted full fine-tune.

Expected: trainable parameters drop by $100\times$ or more; LoRA fits where full fine-tuning OOMs.
Why: only the small low-rank adapters carry gradients and optimizer state.

Lab 2 — Compare Positional Encodings

Reinforces: Part A (RoPE, relative vs absolute, extrapolation).

Hypothesis: relative-style encodings extrapolate beyond the training length; absolute do not.

Procedure: train two tiny models on short sequences — one with learned absolute embeddings, one with RoPE — then evaluate on sequences longer than seen in training. Plot accuracy versus sequence length.

Expected: the absolute model degrades sharply past its training length; RoPE degrades more gracefully. *Why:* relative position generalizes; absolute slots beyond the trained range are undefined.

Lab 3 — Deploy on Kubernetes

Reinforces: Part B (Kubernetes, GPU scheduling).

Hypothesis: a Deployment + Service gives self-healing, load-balanced serving.

```
# Deploy a small model server, then:
kubectl scale deploy/llm -replicas=3
kubectl delete pod <one-pod> # watch it respawn
kubectl get pods -w
```

Procedure: on a local cluster, deploy a small model server as a Deployment behind a Service. Scale replicas, then kill a Pod and watch Kubernetes replace it. Send traffic through the Service and confirm load balancing.

Expected: killed Pods respawn automatically; traffic spreads across replicas. *Why:* the Deployment reconciles actual state to desired state; the Service load-balances.

Lab 4 — Autoscale Under Load

Reinforces: Part B (autoscaling, observability).

Hypothesis: replica count tracks load when scaling on the right signal.

Procedure: configure horizontal autoscaling on queue depth or requests-per-replica. Drive a load ramp and record replicas, queue depth, and p99 latency over time. Note the cold-start lag when scaling up.

Expected: replicas rise with load; p99 stays bounded once warm; a visible lag while new replicas load weights. *Why:* autoscaling reacts to the signal, but model loading delays readiness — the warm-headroom lesson.

Lab 5 — Trace a RAG Request

Reinforces: Part B (observability, distributed RAG).

Hypothesis: a distributed trace localizes latency to a specific stage.

```
# Instrument each stage with OpenTelemetry spans:
with tracer.start_as_current_span("retrieve"): ...
with tracer.start_as_current_span("rerank"): ...
with tracer.start_as_current_span("generate"): ...
# View the waterfall in Jaeger.
```

Procedure: instrument a multi-stage RAG service with OpenTelemetry, send requests, and inspect the trace waterfall in Jaeger. Deliberately slow one stage and confirm the trace pinpoints it.

Expected: the waterfall shows each stage's duration; the slowed stage stands out immediately.
Why: traces attribute latency to its true source — the incident-mindset lesson in practice.

Closing: The Complete Arc

Three books, one continuous explanation.

Book 1 — the model: attention → inference → sampling → RAG → evaluation → serving → reasoning



Book 2 — the hardware: GPU → CUDA → training → engine internals → kernels → cost



Book 3 — the breadth & the platform: architectures → training methods → Kubernetes → serving → ops

Book 1 explained why the model behaves as it does. Book 2 explained why the hardware behaves as it does. Book 3 widened the model story to the full architecture and training landscape, then carried the whole stack up to the production platform where it actually runs.

One idea threads all three: *the binding constraint is memory, bandwidth, and communication — almost never raw compute*. It appears as the KV cache and Flash Attention in Book 1, as HBM, tiling, and FSDP in Book 2, and as gradient checkpointing, GPU scheduling, and warm-replica autoscaling in Book 3. The same move — keep data close, move it less, keep every processor fed — recurs from a single attention head to a cluster of GPU nodes.

An engineer who holds this entire arc, from a token to a production platform, and who has run the labs to feel each layer, does not merely use LLM systems. They can design, optimize, and operate them.